

GPU 性能分析：Nsight Systems 与 Nsight Compute

目 录

1 引言：GPU 优化要分两个层次	2
2 Nsight Systems：全程序时间线	2
2.1 基本命令	2
2.2 时间线上看什么	2
2.3 典型时间线模式	3
3 Nsight Compute：单 kernel 深入分析	3
3.1 基本命令	3
3.2 关键指标	3
3.3 SM Occupancy	4
3.4 Achieved vs Theoretical Occupancy	4
3.5 Register Pressure	4
3.6 Memory Coalescing 与 Throughput	5
3.7 Warp Divergence	5
4 在 ABEGPU 中应用	5
5 关键原则：不要只看单 kernel 加速	5
6 Profiling 闭环	6
7 本章你将学会	7
8 要点速查	8
9 小结	8

1 引言：GPU 优化要分两个层次

上一章我们用 VTune 看清了 host 端调用链，也踩到了“host 等待不等于 kernel 时间”的坑。要回答“GPU 到底在算什么、算了多久、算得有没有效率”，必须用 NVIDIA 自家的 Nsight 工具。GPU profiling 和 CPU profiling 有一个关键区别：它天然分两个层次。

第一个层次是**全局时间线**：整个程序跑下来，哪些 kernel 什么时候发射、host 和 device 之间拷了多少数据、CPU 和 GPU 谁在等谁。这层用 **Nsight Systems**（命令 `nsys`）。第二个层次是**单 kernel 内部**：某个热点 kernel 在 SM（Streaming Multiprocessor）上跑得满不满、内存带宽吃没吃满、寄存器够不够用。这层用 **Nsight Compute**（命令 `ncu`）。

直觉 把 GPU 程序想成一条流水线车间。`nsys` 是车间的“全天录像”，告诉你几点几分哪台机器开了、哪台机器空着、原材料（数据）在传送带上堵了多久。`ncu` 是对某一台机器的“拆解报告”，告诉你这台机器的电机（SM）负载到百分之几、进料口（内存带宽）是不是堵了。先看录像定位哪台机器最忙，再对它出拆解报告，这是 profiling 的正确顺序。

2 Nsight Systems：全程序时间线

Nsight Systems（`nsys`）抓取的是带时间戳的事件流，生成一份包含 CPU 线程、CUDA stream、kernel 发射、内存拷贝、CUDA API 调用的完整时间线。它是 GPU profiling 的“第一站”：在深入任何单 kernel 之前，先看清全局。

2.1 基本命令

```
nsys profile -o report ./run.sh
```

`-o report` 指定输出文件名（生成 `report.nsys-rep`）。默认会抓 CUDA API、kernel、内存拷贝、NVTX 等事件。如果程序用 MPI 启动，需要让 `nsys` 跟着 `mpirun` 走：

```
nsys profile -o report -- mpirun -np 4 ./ABEGPU
```

多 *rank* 时 `nsys` 会为每个进程生成一份报告（如 `report.1.nsys-rep`、`report.2.nsys-rep`），下载到本地用 *Nsight Systems GUI* 分别打开比较。也可以加 `--trace=cuda,nvtx,mpi` 精确控制要抓哪些事件，减少开销。

2.2 时间线上看什么

打开 `nsys` 报告后，时间线视图能回答下面几个问题：

1. **kernel 执行顺序与时间占比**：每个 kernel 是一行横向条带，宽度就是执行时间。一眼能看出哪个 kernel 最宽（最耗时）。
2. **host-device 拷贝**：`cudaMemcpy` 事件显示在时间线上，能看出拷贝是否和 kernel 计算重叠，还是串行阻塞。
3. **CPU 等待 GPU / GPU 等待 CPU**：如果 CPU 线程长时间空着不动，说明它在 `cudaDeviceSynchronize` 上等 GPU；如果 GPU 时间线上有大段空白，说明 GPU 在等 CPU 发射新 kernel。
4. **MPI / CUDA API 串行阻塞**：API 调用如果排成一条长队，说明发射成了瓶颈。

5. **launch 碎片化**: 大量极短的 kernel 紧挨着发射, 每个都有一点 launch overhead, 累积起来很可观。
6. **GPU 空闲区间**: GPU 时间线上的空白就是浪费的算力, 通常意味着 host 控制流没及时提交工作。

直觉 nsys 时间线最大的价值是“看见时序”。采样能告诉你“谁占比最大”, 但“谁先谁后、谁在等谁”只有时间线能回答。比如你发现 ghost zone 交换前有一个 `cudaDeviceSynchronize` 把所有 kernel 排空, 紧接着 `MPI_Waitall` 又在等别的 rank, 这两段等待是串行的, 时间线上一眼就能看出来能不能重叠。

2.3 典型时间线模式

在 ABEGPU 这样的程序里, 一个演化步的时间线通常长这样:

```
CPU 线程: ■ launch bssn_rhs_gpu → ■ sync → ■ MPI_Waitall → ■ launch diff_new_gpu

GPU stream:    [bssn_rhs_gpu 运行]    空闲    [diff_new_gpu 运行]

cudaMemcpy:    ← ghost zone 拷贝 →
```

Figure 1: 一个演化步的 nsys 时间线模式

从这个模式里你能一眼看出三件事: GPU 中间的“空闲”是不是太长, ghost zone 拷贝能不能和计算重叠, CPU 的 sync 等待是否阻塞了下一个 kernel 的发射。

3 Nsight Compute: 单 kernel 深入分析

nsys 定位出最热的那个 kernel 之后, 用 Nsight Compute (ncu) 对它做“体检”。ncu 会运行这个 kernel 并采集 SM 内部的详细计数器, 告诉你它为什么慢。

3.1 基本命令

```
ncu --set full -k kernel_name -o report ./run.sh
```

`--set full` 采集所有可用指标, `-k kernel_name` 只分析名字匹配的 kernel, `-o report` 输出到 `report.ncu-rep`。也可以用 `--kernel-name regex:bssn` 匹配名字含 `bssn` 的所有 kernel。

ncu 默认会“重放”kernel (kernel replay), 即让 kernel 跑很多遍以采集不同计数器组, 开销很大。先用 nsys 定位到唯一的热点 kernel, 再用 `-k` 精确指定它, 避免对全程序所有 kernel 都做 ncu (那样可能跑几十分钟)。

3.2 关键指标

ncu 报告里指标很多, 但最关键的有以下几类:

指标	含义	异常说明
SM Occupancy	实际活跃 warp 数 / 最大 warp 数	低则说明寄存器或 shared memory 限制了并发
Global Memory Throughput	全局内存带宽利用率	接近峰值说明 memory-bound
Memory Coalescing	访问合并程度	低则浪费带宽
Register Per Thread	每线程寄存器数	高则挤压 occupancy
Shared Memory	shared memory 使用量与 bank conflict	bank conflict 串行化访问
Warp Divergence	分支发散比例	高则 warp 内线程各走各路

Table 1: ncu 关键指标

下面把几个最重要的指标展开讲。

3.3 SM Occupancy

Occupancy（占用率）是 SM 上实际活跃的 warp 数占最大 warp 数的比例。V100 每个 SM 最多 64 个 warp（2048 线程）。但实际能跑多少，受三个资源限制：寄存器、shared memory、线程数上限。

直觉 Occupancy 不是“算得快不快”，而是“SM 上能不能塞下足够的 warp 让调度器有东西可换”。如果 occupancy 低，当某个 warp 在等内存，调度器找不到别的 warp 来填空，SM 就空转。但这不绝对：如果一个 warp 本身计算密集到不需要切换，低 occupancy 也能跑满。所以 occupancy 要和吞吐量、延迟一起看。

3.4 Achieved vs Theoretical Occupancy

ncu 会给两个值：**Theoretical Occupancy**（理论占用率，由资源限制算出）和 **Achieved Occupancy**（实际达到的占用率）。如果两者差距大，说明问题在运行时调度（比如 launch 配置不对、kernel 内部分支导致 warp 提前退出）；如果两者接近但都很低，说明问题在资源限制（寄存器或 shared memory 用太多）。

情况	含义	优化方向
理论高 / 实际低	调度或分支问题	检查 launch 配置、warp divergence
理论低 / 实际低	资源限制	减寄存器或 shared memory
理论高 / 实际高	occupancy 不是瓶颈	看 throughput 或 compute

Table 2: 理论 vs 实际 occupancy 的诊断

3.5 Register Pressure

每线程用的寄存器越多，一个 SM 能容纳的线程就越少，occupancy 就越低。编译器会自动分配寄存器，但你可以通过 `__launch_bounds__` 或编译选项（如 `--maxrregcount`）强制限制，代价是寄存器溢出（spill）到 local memory，反而变慢。

寄存器优化是一个“走钢丝”：减寄存器能提高 *occupancy*，但可能引入 *spill* 反而变慢。每次改完都要重跑 *ncu* 看是否 *spill* 增加、*occupancy* 是否真的提升。

3.6 Memory Coalescing 与 Throughput

Memory Coalescing（合并访问）指一个 warp 内 32 个线程的访存地址能否合并成少数几次大事务。如果 32 个线程各访问不相邻的地址，GPU 要做 32 次独立事务，带宽利用率暴跌。**Global Memory Throughput** 则是实测的带宽与峰值带宽的比值，接近 100% 说明 kernel 已经在吃满内存带宽，是 memory-bound。

直觉 合并访问像“32 个人一起坐大巴”（一次大事务），不合并像“32 个人各打一辆出租车”（32 次小事务）。大巴一次运完，出租车要跑 32 趟。访存模式决定你用的是大巴还是出租车。

3.7 Warp Divergence

GPU 以 warp（32 线程）为单位执行，同一个 warp 内的线程必须走同一条指令。如果遇到 if-else 分支，一部分线程走 if、一部分走 else，就只能串行执行两遍，这就是**分支发散**（warp divergence）。*ncu* 报告分支发散比例，高的话考虑用掩码或重排数据消除分支。

4 在 ABEGPU 中应用

把 *nsys* 和 *ncu* 套到 ABEGPU 上，推荐的流程是：

1. 用 ***nsys*** 看整个演化流程：先跑一次完整 baseline，在时间线上看 *bssn_rhs_gpu*、*diff_new_gpu*、*prolongrestrict* 等 kernel 各占多少时间，哪个最宽就是热点。
2. 检查 **host-device 拷贝**：ghost zone 交换前后是否有不必要的 *cudaMemcpy*，能否用 pinned memory 或 stream 重叠隐藏。
3. 检查 **ghost exchange 通信开销**：MPI_Waitall 和 kernel 之间的时序关系，能否把通信和计算重叠。
4. 用 ***ncu*** 深入最热 kernel：对最宽的那个 kernel 跑 *ncu --set full*，判断它是 compute-bound 还是 memory-bound。
5. 判断瓶颈类型：throughput 接近峰值是 memory-bound，occupancy 低且 throughput 低多半是 register / launch 配置问题。
6. 优化后重新 **profile**：改完代码后重跑 *nsys* 和 *ncu*，确认热点比例下降、occupancy 上升。

ABEGPU 中常见的 kernel 有 *bssn_rhs_gpu*（BSSN 右端项计算）、*diff_new_gpu*（差分更新）、*prolongrestrict*（网格 *prolongation* 和 *restriction*）。先用 *nsys* 确认哪个占比最大，通常 *bssn_rhs_gpu* 会是头号热点，因为它每步都跑且计算量最大。

5 关键原则：不要只看单 kernel 加速

这是 GPU 优化最容易犯的错。你把 *bssn_rhs_gpu* 优化快了 3 倍，兴奋地重新跑端到端，结果发现总时间只快了 10%。为什么？因为整个程序不只有这一个 kernel，还有大量小

kernel、host 控制流、MPI 通信、host-device 拷贝。单 kernel 快了，可能引入了额外的同步、拷贝、launch 开销，反而把别的地方拖慢。

直觉 把整个程序想成一条管道，每个 kernel 是管道上的一段。你把某一段加粗了 3 倍，但如果管道最窄的口（瓶颈）在别处，总流量由最窄口决定，加粗那段不会让总流量变快。这就是 Amdahl 定律的直觉：只有优化占比最大的瓶颈，端到端才有明显提升。

所以判断优化是否有效，必须看**端到端**时间，而不是单 kernel 时间。nsys 的时间线能帮你确认“优化后哪个 kernel 变宽了、哪个变窄了、有没有引入新的等待”。

一个常见的反优化陷阱：为了减少 kernel 数量，把两个 kernel 合并成一个，结果中间多了一次 host 同步，或者合并后的 kernel 寄存器压力暴涨导致 occupancy 暴跌，端到端反而变慢。这种问题只有看 nsys 时间线才能发现。

6 Profiling 闭环

把上面所有环节串起来，GPU 优化的标准闭环是：

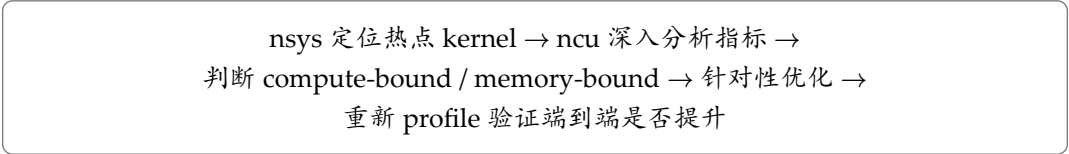


Figure 2: GPU profiling 闭环

这个闭环要反复执行：每次优化后回到 nsys，确认旧的热点是否下降、有没有冒出新的热点，再决定下一轮优化方向。

例 假设我们在 ABEGPU baseline 上用 nsys 定位到 bssn_rhs_gpu 是最热 kernel，接着用 ncu 采集到以下关键指标：

指标	数值
Theoretical Occupancy	40%
Achieved Occupancy	39%
Register Per Thread	80
Global Memory Throughput	35%

Table 3: bssn_rhs_gpu 的 ncu 结果（假设）

先看 occupancy：V100（compute capability 7.0）每个 SM 最多 64 个 warp、2048 线程，寄存器总数 65536，分配粒度每 warp 256 个。设每线程寄存器数 $R = 80$ ，则每 warp 寄存器数为

$$32 \times 80 = 2560 \tag{6.1}$$

按 256 粒度取整后仍是 2560，那么寄存器限制下的最大 warp 数为

$$\text{floor}(65536/2560) = 25 \tag{6.2}$$

对应线程数 $25 \times 32 = 800$ ，理论 occupancy 为

$$800/2048 \approx 39\% \quad (6.3)$$

这与 ncu 报告的 Theoretical Occupancy 40% 基本一致，而 Achieved Occupancy 39% 也几乎等于理论值。这说明 occupancy 的瓶颈不在运行时调度，而在寄存器压力：每线程 80 个寄存器把 SM 塞满了。

再看 Global Memory Throughput 只有 35%，远未到峰值，所以这个 kernel 不是 memory-bound，而是 occupancy 受限导致 SM 上没有足够 warp 来隐藏访存延迟，吞吐量上不去。

优化方向因此很清楚：减少每线程的寄存器使用。若把 R 从 80 降到 64：

$$32 \times 64 = 2048, \quad \text{floor}(65536/2048) = 32 \quad \text{warps} \quad (6.4)$$

$$32 \times 32 = 1024, \quad 1024/2048 = 50\% \quad (6.5)$$

若进一步降到 $R = 40$ ：

$$32 \times 40 = 1280, \quad \text{floor}(65536/1280) = 51 \quad \text{warps} \quad (6.6)$$

$$51 \times 32 = 1632, \quad 1632/2048 \approx 80\% \quad (6.7)$$

把寄存器从 80 降到 40，理论 occupancy 从 39% 升到约 80%，几乎翻倍。具体手段包括用 `__launch_bounds__` 提示编译器、减少同时活跃的局部变量、把部分中间结果存到 shared memory。但要注意寄存器降到一定程度会发生 spill 到 local memory，反而变慢，所以每改一次都要重跑 ncu 验证。

7 本章你将学会

1. 区分 Nsight Systems 和 Nsight Compute 的用途，知道先 nsys 后 ncu 的顺序
2. 运行 `nsys profile` 和 `ncu --set full` 命令采集 GPU profiling 数据
3. 解读 SM Occupancy、Global Memory Throughput、Memory Coalescing、Register Pressure、Warp Divergence 指标
4. 区分 Theoretical Occupancy 与 Achieved Occupancy，定位 occupancy 瓶颈来源
5. 避免单 kernel 思维，用端到端时间判断优化是否有效
6. 在 ABEGPU 中应用“nsys 定位 → ncu 深入 → 优化 → 重新 profile”的闭环

8 要点速查

指标 / 视图	含义	优化方向
nsys 时间线	kernel 时序与 host-GPU 等待	重叠计算与通信
nsys GPU 空白	GPU 空闲区间	及时提交 host 工作
ncu SM Occupancy 低	warp 数不足	减寄存器 / 调 launch 配置
ncu Throughput 接近峰值	吃满带宽	已是 memory-bound, 减数据搬运
ncu Coalescing 低	访问未合并	重排数据布局
ncu Register 高	寄存器挤压 occupancy	__launch_bounds__ 减寄存器
ncu Warp Divergence 高	分支发散	掩码或重排数据

Table 4: GPU profiling 指标速查

9 小结

GPU profiling 分两层: Nsight Systems 看全局时间线, 定位最热 kernel 和 host-GPU 时序问题; Nsight Compute 看单 kernel 内部, 解读 occupancy、throughput、coalescing、register 等微架构指标。正确顺序是先用 nsys 找到最宽的 kernel, 再用 ncu 对它出“体检报告”, 判断是 compute-bound 还是 memory-bound, 针对性优化后再回到 nsys 验证端到端是否真的提升。最忌讳的是只盯单 kernel 加速倍数, 因为程序里还有大量 kernel、host 控制、通信和拷贝, 一个 kernel 快了不代表整体快, 有时甚至引入新开销反而变慢。把 nsys 和 ncu 配合成闭环, 才能让每一轮优化都有可解释的收益。

讲义基于 HPC101 2025 Lab4 实验内容编写